

Why We Need to Measure What Gets Lost Before Retrieval Begins A Practitioner's Framework for the Knowledge Preservation Ratio

Venkat Bhamidipati CTO, Datasakt Technologies

AI FOR BUSINESS & ENGINEERING LEADERS · TECHNICAL DEEP DIVE

Introduction

A previous article introduced the Knowledge Preservation Ratio (KPR) as a corrective to a blind spot in enterprise AI evaluation: every standard retrieval metric assumes a searchable representation already exists, and measures only how well a system retrieves from it. None of them ask the prior question—how much of the original document's knowledge survived being turned into that representation in the first place.

That article left KPR as a concept. This one exists to answer the question a concept cannot avoid forever: how do you actually count it?

KPR is defined as a ratio:

$$\text{KPR} = \frac{\text{Knowledge Preserved After Transformation}}{\text{Knowledge Present in Original}}$$

The formula is trivial. The numerator and the denominator are not. Defining what counts as "knowledge," in a way that two independent people would count the same way, is the entire difficulty—and the entire value, if it can be done. A framework that cannot be measured consistently is not a framework. It is a slogan. This article is an attempt to make KPR something a team can actually compute on its own documents, not just cite.

Part I — Why Existing Metrics Don't Answer This Question

Precision, recall, F1, Mean Reciprocal Rank, NDCG, and hallucination rate are all retrieval-stage or generation-stage metrics. They share a hidden assumption: that the candidate pool being scored already contains a faithful representation of the source document. They ask "did the system find the right chunk" or "did the model's answer match the chunk it found." None of them ask whether the chunk itself still contains the knowledge the original page did.

This is not a minor gap. A system can score well on every standard metric while quietly operating on a representation that lost the table structure, the chart, or the conditional

clause that mattered most. Precision measures fidelity to a corrupted ground truth as faithfully as it measures fidelity to an intact one. The metric cannot tell the difference, because it was never designed to look upstream of retrieval.

KPR is positioned deliberately upstream of where these metrics start. It does not replace them. It answers the question they structurally cannot.

Part II — The Denominator: Defining "Original Knowledge"

This is the harder of the two terms, and the one most attempts at this kind of framework fail to operationalize. "All the knowledge in the document" is not countable. Something narrower and more concrete is needed.

Working definition:

Original Knowledge = the set of semantically distinct, query-relevant units that a domain expert would identify as necessary to correctly answer the domain's most important question types.

Two things make this definition usable rather than philosophical. First, it is bounded by query relevance, not by the document's total content—a footnote about formatting conventions is not a knowledge unit if no realistic query depends on it. Second, it is anchored to a human judgment that can be checked for consistency across judges, rather than to an abstract notion of "everything in the document."

A Typology of Knowledge Units

Counting requires categories. The following typology is proposed as a starting taxonomy—not a closed list, but a set of unit types that in practice covers the large majority of what regulated-domain documents carry.

Unit Type	Definition	Example	Countability
Fact	A discrete, verifiable statement	"Drug A dosage = 10mg"	Direct count
Observation	A measured or observed value or trend	"Survival declined after week 6"	Count, with a threshold for what counts as a distinct observation
Relationship	A connection between two or more entities	"Drug A treats Disease B"	Direct count
Constraint	A condition that limits when a fact or relationship applies	"Only valid for patients over 65"	Direct count
Evidence	Quantitative support attached to a claim	" $p < 0.01$, 95% CI: 0.62–0.84"	Direct count
Visual Trend	A pattern expressed only in non-text media	"Chromatography peak anomaly at 4.2 min"	Countable via expert annotation of the image, not the caption

This list is intentionally not exhaustive. Structural references (cross-document pointers, "see Section 4.2") and temporal conditions (facts that are only true as of a given date or protocol version) are two further categories that matter in regulatory contexts and can be added as a domain requires. The taxonomy should be treated as extensible, with the constraint that any added category must meet the same bar: a domain expert can point to an instance of it in a document and say, unambiguously, "that is one of these."

Establishing the Denominator in Practice

For a given document and a given set of target query types, the denominator is produced through expert annotation, not automatic extraction. A domain expert (or, more robustly, two experts working independently) reads the source document and tags every instance of each unit type that a realistic query in that domain could depend on. The count of tagged units is the denominator.

This is deliberately a human number, not a machine-computed one. Automating denominator extraction would reintroduce exactly the problem KPR exists to catch: a flawed extraction process silently deciding what counts as knowledge, the same failure mode the framework is built to detect downstream.

Part III — The Numerator: Defining "Preserved Knowledge"

Working definition:

Preserved Knowledge = the subset of denominator units that can be fully and correctly recovered from the transformed representation, without loss of meaning, by someone who has access only to that representation—not the source document.

The operative test is a recovery test, and it has a precise form: take the system's stored representation (chunks, embeddings, graph triples, whatever the architecture produces) and ask a human reviewer, or a sufficiently capable model given only that representation, to reconstruct each denominator unit. If the unit can be reconstructed exactly, it is preserved. If it cannot be reconstructed at all, it is lost. The harder, and more honest, case is in between.

Scoring Partial Recovery

Binary preserved-or-lost scoring undercounts what actually happens in practice. A chunked dissolution table might allow a reviewer to recover that dissolution increased over time, without allowing recovery of the exact percentage at the 60-minute mark or the specification limit it should be compared against. That is not full preservation, and it is not total loss either.

Recovery Outcome	Definition	Score
Full recovery	Unit reconstructed exactly, including all qualifying detail	1.0
Partial recovery	Core fact or direction recovered; magnitude, conditions, or precision lost	0.5
No recovery	Unit cannot be reconstructed from the representation at all	0.0

The 0.5 midpoint is a deliberately simple starting convention, not a precise measurement. Teams with more annotation budget can use a finer scale; the discipline that matters is that partial recovery is scored as partial, not rounded up to full or down to zero.

Who Performs the Recovery Test, and Why That Choice Matters

There are two valid ways to run the recovery test, and they answer different questions.

A human reviewer performing the recovery test measures what the representation makes available to a human auditor—the relevant question for compliance review, manual QA, or any workflow with a human in the loop.

A capable LLM given only the stored representation, with no access to the source document, performing the recovery test measures something different and equally important: what the production system itself can actually answer with, since that LLM's access mirrors what the deployed RAG or Graph RAG pipeline gives the model at inference time. Running both, on the same annotated sample, and comparing the gap between them surfaces a specific and useful failure signature: a representation that a human could reconstruct from but a model could not is a prompting or context-formatting problem. A representation that neither can reconstruct from is a transformation problem further upstream, and no amount of prompt engineering will fix it.

Part IV — The Core Formula

Unweighted KPR, using the scoring above:

$$\text{KPR} = \frac{\sum (\text{Recovery Score for each denominator unit})}{\text{Total Number of Denominator Units}}$$

This treats every knowledge unit as equally important. In most regulated domains, that assumption is false, and pretending otherwise produces a number that is measurable but misleading.

Weighted KPR corrects for this by attaching an importance weight to each unit type before aggregating:

$$\text{Weighted KPR} = \frac{\sum (\text{Recovery Score}_i \times \text{Weight}_i)}{\sum (1.0 \times \text{Weight}_i, \text{ for every denominator unit } i)}$$

Illustrative Weighting Scheme

The following weights are an illustrative scenario for a pharmaceutical safety use case, included to show the mechanism—not a benchmarked or industry-standard weighting. Any team adopting this approach should derive its own weights from its own domain's actual risk consequences, ideally with input from the relevant regulatory or quality function.

Knowledge Type	Illustrative Weight	Rationale
Page or footnote reference	1	Low consequence if lost
Figure caption	2	Low to moderate consequence
Dosage or strength value	10	Direct clinical consequence
Statistical significance / evidence	15	Governs whether a claim is even valid
Contraindication	30	High safety consequence
Safety warning	50	Highest safety consequence

The reason weighting matters is not cosmetic. A system that preserves every safety warning but loses every figure caption should score well on weighted KPR and poorly on unweighted KPR—and both of those outcomes are correct for a safety-critical domain. A single unweighted number flattens a distinction that matters enormously: not all lost knowledge is equally dangerous to lose.

Part V — A Measurement Protocol

The following is a working protocol a team can run against its own corpus, scoped to be completed in a small number of person-weeks rather than as an open-ended research project.

Step	Action	Effort
1	Select 50–100 representative documents spanning the corpus's main document types	Low
2	Define the 10–20 query types that matter most for the domain	Medium
3	Have two domain experts independently annotate denominator units on a sample of 5–10 documents	Medium
4	Compute inter-annotator agreement (Cohen's κ); resolve disagreements and refine the unit typology if agreement is low	Medium
5	Run the full document sample through the actual production pipeline (OCR, chunking, embedding, graph extraction, or whatever the architecture under test uses)	Low
6	For each annotated unit, run the recovery test against the resulting representation and assign a recovery score	Medium
7	Compute unweighted and weighted KPR from the resulting scores	Low

A realistic estimate for a mid-sized enterprise team running this for the first time, on a single document type, is two to three person-weeks—most of it concentrated in Steps 3 and 6, which are inherently manual. Subsequent runs, once the unit typology and weighting scheme are established, are substantially faster, since Steps 1, 2, and 4 do not need to be repeated from scratch.

Step 4 deserves emphasis. If two domain experts cannot agree on what counts as a knowledge unit—if Cohen's κ comes in low—the problem is not the annotators. It is that the unit typology is too vague for the domain in question and needs to be tightened before any KPR number produced from it can be trusted. A KPR score computed on top of poor inter-annotator agreement is not a measurement. It is noise wearing the shape of a metric.

When to Measure KPR

The protocol above raises an obvious question: is this realistic for a live enterprise project, or only as a one-time academic exercise? The answer depends on treating KPR as a design-time and validation-time framework, not a runtime one.

KPR is not meant to run on every query, the way latency or precision@k does. It belongs alongside a different category of engineering activity—closer to a crash-test rating than a speedometer. You do not crash-test every car that leaves the factory; you crash-test the

design, then monitor production for signals that something has changed. KPR works the same way, and it fits naturally into three recurring moments in a project's lifecycle.

Before deploying a new pipeline on a document type. This is the primary use case and the one the protocol above is scoped for: establish a KPR baseline before the system goes live, while a low number is still a design problem rather than a production incident.

After changing extraction, chunking, or graph construction. Once a typology and weighting scheme already exist for a document type, a regression check is far cheaper than the first measurement—often a matter of days rather than weeks, since only the pipeline re-run and recovery test (Steps 5-7) need repeating, not the annotation and agreement work of Steps 2-4.

During incident investigation. When a specific failure surfaces—a missed contraindication, a regulatory finding—a targeted KPR analysis on the document in question, and a handful of similar ones, can usually identify the exact transformation step that destroyed the relevant unit, often within a day.

What KPR is not intended for is continuous, per-query computation. Attempting that would misread both the cost of the framework and the question it is built to answer. The honest comparison is not "KPR versus free." It is "the cost of measuring before deployment versus the cost of discovering KPR was 0.3 after a compliance failure already happened."

Part VI — An Illustrative Benchmarking Scenario

The table below is a constructed scenario, intended to show how KPR results might look once measured, not a result drawn from a specific benchmarked study. Treat the numbers as rounded ranges and as illustrative of relative ordering, not as precise findings to cite.

Architecture	Unweighted KPR (illustrative range)	Weighted KPR, safety-critical (illustrative range)	What drives the gap
Text-only RAG	0.30-0.40	0.25-0.35	Loses images, table structure, and most constraints
Hybrid RAG (BM25 + dense)	0.45-0.55	0.40-0.50	Better retrieval precision; same upstream extraction losses
Graph RAG	0.45-0.55	0.25-0.35	High recovery on entities and relationships; low recovery on the conditions and evidence attached to them
Multimodal RAG (VLM parsing + table extraction)	0.65-0.75	0.60-0.70	Recovers most visual and tabular content; scientific charts remain inconsistent
Human expert review	0.90+	0.90+	Effective upper bound; not scalable

The pattern worth noticing is not any single number—it is the gap between unweighted and weighted KPR for Graph RAG specifically. A graph can preserve the entity-relationship skeleton of a clinical finding while discarding almost everything that makes the finding safe to act on. A KPR measurement that only reported the unweighted column would miss the most important thing this scenario illustrates.

Part VII — A Research Question Worth Naming

The protocol above is workable today, but it raises a question that is genuinely open and worth stating as a research agenda rather than a solved problem:

What is the smallest set of objectively measurable knowledge-unit types that allows KPR to be computed consistently across text, tables, images, and graph representations?

A testable hypothesis: a taxonomy of seven to eight unit types—fact, observation, relationship, constraint, evidence, visual trend, structural reference, and temporal condition—covers the substantial majority of query-relevant knowledge in regulated enterprise documents, across pharma, clinical, financial, and manufacturing domains.

A testable claim that would validate it: two independent annotators, using this taxonomy and given basic training, achieve Cohen's κ above 0.8 on representative document samples across at least three distinct domains.

Either of these could be tested directly by any team or research group with access to a modest annotation budget and a willingness to publish the inter-annotator agreement numbers, including the negative result if κ falls short.

Part VIII — Limitations, Stated Plainly

A framework that hides its own weaknesses is not more credible for doing so. KPR has real limits.

It does not collapse to a single number that is valid for all queries against a document—different query types may legitimately weight the same units differently, which means a single KPR score should always be reported alongside the query types it was computed for, not presented as a universal property of a document or pipeline.

It costs real annotation time. Two to three person-weeks per document type is a genuine investment, not a free metric a team gets from existing logs. The honest comparison is not against zero cost; it is against the cost of discovering a representation's failure after it has already caused a production incident.

It measures recoverability from a representation, not end-to-end retrieval quality. A representation can have a high KPR and still be retrieved poorly due to ranking, indexing, or query-formulation problems further downstream. KPR and retrieval metrics answer different questions and are meant to be used together, not as substitutes for each other.

And for very large corpora, exhaustive annotation is not feasible. Sampling is required, which means KPR, like any sampled metric, carries a margin of error that should be reported, not assumed away.

Conclusion

The earlier article asked what knowledge is lost when enterprise documents are transformed into searchable representations. This one asks the harder, necessary follow-up: how is that loss measured consistently enough for the answer to mean something beyond one person's intuition?

KPR does not replace precision, recall, or F1. It answers a question those metrics were never built to ask: how much of the source knowledge survived before retrieval even began. A system can report excellent precision while operating on a representation with a KPR of

0.30. That is not a contradiction. It is precision measured faithfully against a foundation that was already hollowed out.

In a regulated enterprise context, a system with 95 percent retrieval precision and 30 percent KPR is not a system with one good score and one bad score to improve later. It is a system that retrieves very reliably from a representation that has already discarded most of what mattered—precisely calibrated against the wrong target.

The organizations that learn to measure KPR before deployment will stop being surprised when a polished demonstration fails in production. The ones that do not will keep discovering, after the fact, exactly what their pipeline quietly decided was not worth keeping.

Further Reading

This article is a practitioner framework rather than a formal academic paper, and the measurement protocol above should be treated as a starting point for a team's own validation, not a finished standard. Readers interested in the underlying material this builds on may find the following useful starting points: standard information retrieval evaluation literature (precision, recall, NDCG, MRR) for the metrics KPR is positioned alongside rather than against; the inter-annotator agreement literature (Cohen's κ and its extensions) for rigor in Step 4 of the measurement protocol; and the broader knowledge representation and information loss literature for the theoretical grounding behind the original critique this framework was built to operationalize.

Venkat Bhamidipati is CTO and Co-founder of Datasekt Technologies, Bengaluru, an enterprise AI product company specialising in document intelligence, OCR, and intelligent automation. With over 30 years of experience spanning telecom infrastructure, enterprise software, and AI product development, he works at the intersection of knowledge representation, regulatory compliance, and production-scale AI deployment across pharma, banking, and manufacturing domains. He is the author of seven books in the AI for Business Managers series.